

Agente de Llamadas — Asterisk

Agente de Control de Llamadas — Asterisk

Documento técnico de referencia para el equipo de desarrollo. Última actualización: mayo 2025

Índice

1. Resumen del sistema
2. Stack tecnológico
3. Dependencias y versiones
4. Arquitectura de interfaces
5. Requisitos del servidor
6. Instalación paso a paso
7. Configuración mínima de Asterisk
8. Estructura del proyecto
9. Variables de entorno
10. Arranque del sistema
11. Verificación y pruebas
12. Notas para el equipo

1. Resumen del sistema

El sistema implementa un **agente de control de llamadas** sobre Asterisk PBX que permite:

- Originar, monitorear, transferir y colgar llamadas de forma programática.
- Escuchar eventos de canal en tiempo real (marcado, contestación, cuelgue, DTMF).
- Ejecutar lógica de negocio en cada llamada (IVR inteligente, routing, validación contra BD).
- Registrar CDR (Call Detail Records) en base de datos relacional.
- Exponer una API REST interna para que otros módulos del sistema disparen acciones sobre llamadas.

2. Stack tecnológico

Capa	Tecnología	Rol
PBX	Asterisk 20 LTS	Motor de telefonía

Capa	Tecnología	Rol
Canal SIP	chan_pjsip	Troncales y extensiones
Interfaz de control	AMI (Asterisk Manager Interface)	Eventos + acciones en tiempo real
Lógica por llamada	FastAGI	Scripts externos en el flujo del dialplan
API moderna (opcional)	ARI (Asterisk REST Interface)	Control granular de canales vía REST+WebSocket
Backend del agente	Python 3.9+	Proceso daemon que conecta con AMI/AGI
Librería AMI	panoramisk	Ciente AMI asíncrono para Python
Librería AGI	pyst2	Protocolo AGI para Python
Base de datos	PostgreSQL 14+	CDR, configuración, logs de llamadas
ORM	psycopg2 / SQLAlchemy	Acceso a BD desde el agente
Cola de tareas (opcional)	Celery + Redis	Procesamiento asíncrono post-llamada
Servidor web interno	FastAPI	Endpoints REST para disparar acciones
Proceso supervisor	systemd	Gestión del daemon del agente
Logging	logging + loguru	Trazas estructuradas

3. Dependencias y versiones

Sistema operativo

Ubuntu 22.04 LTS (recomendado)
CentOS 7 / AlmaLinux 8 (compatible, ajustar gestor de paquetes)

Asterisk

Asterisk 20 LTS (rama estable de largo soporte)
pjproject 2.13+ (requerido por chan_pjsip)

No usar Asterisk 16/18 — ya están en fin de vida útil.

Python

```
Python >= 3.9  
pip >= 23
```

Paquetes Python (requirements.txt):

```
panoramisk==2.0.0      # cliente AMI asíncrono  
pyst2==0.5.1          # protocolo AGI  
psycopg2-binary==2.9.9 # PostgreSQL  
SQLAlchemy==2.0.23    # ORM (opcional)  
fastapi==0.110.0      # API REST interna  
uvicorn[standard]==0.27 # servidor ASGI  
python-dotenv==1.0.1  # variables de entorno  
loguru==0.7.2         # logging estructurado  
celery==5.3.6         # cola de tareas (opcional)  
redis==5.0.1          # broker Celery (opcional)  
aiohttp==3.9.3        # requests asíncronos
```

Base de datos

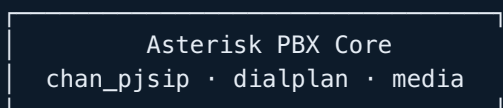
```
PostgreSQL >= 14
```

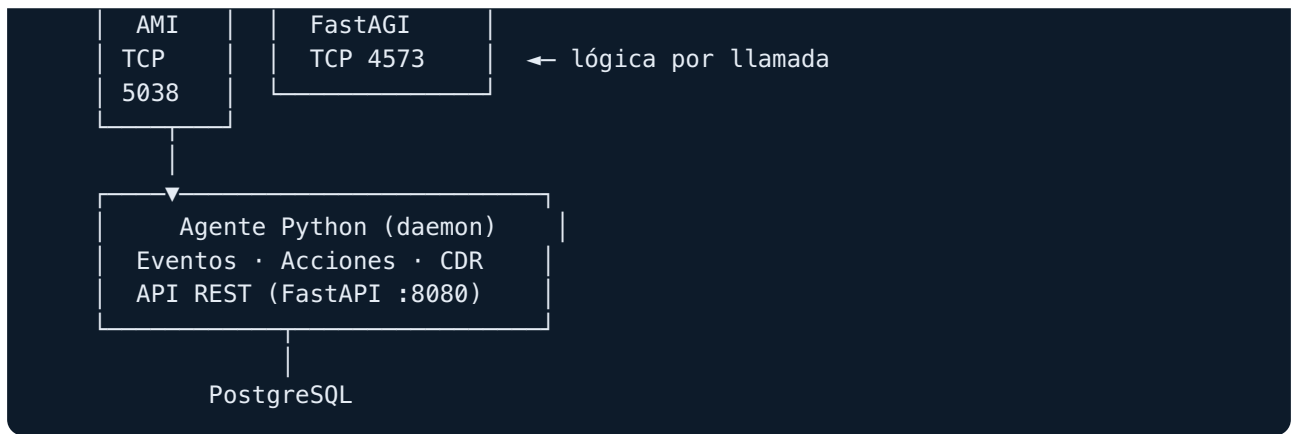
Paquetes del sistema (apt)

```
asterisk asterisk-dev  
asterisk-config  
libasterisk-agi-perl  # si se usan scripts Perl legacy  
python3.9-dev  
python3-pip  
python3-venv  
postgresql-14  
redis-server          # solo si se usa Celery  
git  
build-essential
```

4. Arquitectura de interfaces

PSTN / SIP / WebRTC





Criterio de selección de interfaz

Necesidad	Interfaz a usar
Escuchar eventos de todas las llamadas	AMI
Originar llamadas desde el backend	AMI (Originate)
Colgar / transferir canales	AMI (Hangup , Redirect)
IVR con lógica de negocio en el flujo	FastAGI
Control granular de media (grabar, mezclar)	ARI
Integración con apps de terceros	API REST interna (FastAPI)

5. Requisitos del servidor

Recurso	Mínimo	Recomendado
CPU	2 vCPU	4 vCPU
RAM	2 GB	4 GB
Disco	20 GB	50 GB (grabaciones)
Red	100 Mbps	1 Gbps
OS	Ubuntu 22.04	Ubuntu 22.04 LTS
Puertos abiertos	5060 UDP/TCP (SIP), 10000-20000 UDP (RTP), 5038 TCP (AMI — solo interno), 8080 TCP (API)	ídem

Importante: Los puertos AMI (5038) y ARI (8088) **nunca deben exponerse a Internet**. Solo acceso desde `127.0.0.1` o red interna.

6. Instalación paso a paso

6.1 Asterisk

```
# Dependencias del sistema
sudo apt update && sudo apt install -y \
    build-essential wget curl git \
    libssl-dev libncurses5-dev libnewt-dev \
    libsqlite3-dev uuid-dev libjansson-dev \
    libxml2-dev libxslt1-dev \
    python3-dev python3-pip python3-venv

# Descargar y compilar Asterisk 20
cd /usr/src
sudo wget https://downloads.asterisk.org/pub/telephony/asterisk/asterisk-20-
    current.tar.gz
sudo tar -xzf asterisk-20-current.tar.gz
cd asterisk-20*/

sudo contrib/scripts/install_prereq install
sudo ./configure --with-jansson-bundled
sudo make menuselect      # activar: res_pjsip, app_queue, cdr_custom, res_agi
sudo make -j$(nproc)
sudo make install
sudo make samples         # copia configs de ejemplo a /etc/asterisk/
sudo make config          # instala servicio systemd

sudo systemctl enable asterisk
sudo systemctl start asterisk
```

6.2 Entorno Python del agente

```
# Crear usuario de servicio
sudo useradd -r -s /bin/false asterisk-agent

# Clonar repositorio del agente
git clone <repo_url> /opt/call-agent
cd /opt/call-agent

# Entorno virtual
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

6.3 PostgreSQL

```
sudo apt install -y postgresql-14

# Crear BD y usuario
```

```
sudo -u postgres psql <<EOF
CREATE USER call_agent WITH PASSWORD 'cambiar_password';
CREATE DATABASE call_agent_db OWNER call_agent;
GRANT ALL PRIVILEGES ON DATABASE call_agent_db TO call_agent;
EOF
```

7. Configuración mínima de Asterisk

/etc/asterisk/manager.conf

```
[general]
enabled = yes
port = 5038
bindaddr = 127.0.0.1
displayconnects = no

[call_agent]
secret = CAMBIAR_PASSWORD_SEGURO
permit = 127.0.0.1/255.255.255.255
read = all
write = all,originate
```

/etc/asterisk/pjsip.conf (extracto mínimo)

```
[transport-udp]
type = transport
protocol = udp
bind = 0.0.0.0:5060

[troncal_principal]
type = endpoint
transport = transport-udp
context = from-trunk
disallow = all
allow = ulaw,alaw
aors = troncal_principal

[troncal_principal]
type = aor
contact = sip:proveedor.ejemplo.com:5060

; Extensiones internas (ejemplo)
[1001]
type = endpoint
context = from-internal
disallow = all
allow = ulaw,alaw
auth = auth1001
aors = aors1001

[auth1001]
```

```

type = auth
auth_type = userpass
username = 1001
password = extensionpass

[aors1001]
type = aors
max_contacts = 1

```

/etc/asterisk/extensions.conf (dialplan)

```

[globals]
AGENTE_IP=127.0.0.1
AGENTE_PORT=4573

[from-internal]
; Llamadas salientes normales
exten => _9X.,1,Set(CDR(userfield)=outbound)
exten => _9X.,n,Dial(PJSIP/${EXTEN:1}@troncal_principal,30)
exten => _9X.,n,Hangup()

; Contexto IVR - pasa por FastAGI antes de enrutar
exten => 7000,1,Answer()
exten => 7000,n,AGI(agi://${AGENTE_IP}:${AGENTE_PORT})
exten => 7000,n,Hangup()

[from-trunk]
; Llamadas entrantes - van al IVR
exten => s,1,Goto(from-internal,7000,1)

```

/etc/asterisk/cdr_pgsql.conf (CDR a PostgreSQL)

```

[global]
hostname=localhost
port=5432
dbname=call_agent_db
user=call_agent
password=CAMBIAR_PASSWORD_SEGURO
table=cdr

```

8. Estructura del proyecto

```

/opt/call-agent/
├─ main.py           # Punto de entrada - inicia AMI listener + FastAGI +
API
├─ requirements.txt
├─ .env             # Variables de entorno (no versionar)
├─ .env.example     # Plantilla de variables
├─
└─ agent/

```

```

├── ami_client.py      # Conexión y handlers de eventos AMI
├── agi_server.py      # Servidor FastAGI (TCP)
├── call_actions.py    # Originate, Hangup, Transfer, Hold
├── event_handlers.py  # Lógica de negocio por evento
├── api/
│   ├── main.py        # Router FastAPI
│   └── routes/
│       ├── calls.py   # POST /calls/originate, DELETE /calls/{channel}
│       └── status.py  # GET /calls/active
├── db/
│   ├── connection.py  # Pool de conexiones PostgreSQL
│   ├── models.py      # Modelos SQLAlchemy
│   └── cdr.py         # Inserción y consulta de CDR
├── config/
│   └── settings.py    # Carga de .env y validaciones
└── systemd/
    └── call-agent.service # Unidad systemd para producción

```

9. Variables de entorno

Crear `/opt/call-agent/.env` basado en `.env.example`:

```

# Asterisk AMI
AMI_HOST=127.0.0.1
AMI_PORT=5038
AMI_USER=call_agent
AMI_SECRET=CAMBIAR_PASSWORD_SEGURO

# FastAGI
AGI_HOST=0.0.0.0
AGI_PORT=4573

# ARI (si se usa)
ARI_HOST=http://127.0.0.1:8088
ARI_USER=agente_ari
ARI_SECRET=CAMBIAR_PASSWORD_ARI

# PostgreSQL
DB_HOST=127.0.0.1
DB_PORT=5432
DB_NAME=call_agent_db
DB_USER=call_agent
DB_PASSWORD=CAMBIAR_PASSWORD_SEGURO

# API interna
API_HOST=0.0.0.0
API_PORT=8080

# Entorno

```

```
DEBUG=false
LOG_LEVEL=INFO
```

10. Arranque del sistema

Desarrollo (manual)

```
cd /opt/call-agent
source venv/bin/activate

# 1. Verificar que Asterisk esté activo
sudo systemctl status asterisk

# 2. Lanzar el agente
python main.py
```

Producción (systemd)

Archivo `/opt/call-agent/systemd/call-agent.service`:

```
[Unit]
Description=Agente de Control de Llamadas Asterisk
After=network.target asterisk.service postgresql.service
Requires=asterisk.service

[Service]
Type=simple
User=asterisk-agent
WorkingDirectory=/opt/call-agent
ExecStart=/opt/call-agent/venv/bin/python main.py
Restart=on-failure
RestartSec=5
EnvironmentFile=/opt/call-agent/.env
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

```
sudo cp systemd/call-agent.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable call-agent
sudo systemctl start call-agent

# Ver logs en vivo
sudo journalctl -u call-agent -f
```

11. Verificación y pruebas

Verificar conexión AMI

```
# Conectar manualmente al AMI
telnet 127.0.0.1 5038

# Autenticar (copiar y pegar):
Action: Login
Username: call_agent
Secret: CAMBIAR_PASSWORD_SEGURO

# Deben llegar eventos. Para salir:
Action: Logoff
```

Originar una llamada de prueba

```
curl -X POST http://localhost:8080/calls/originate \
-H "Content-Type: application/json" \
-d '{
  "channel": "PJSIP/1001",
  "extension": "1002",
  "context": "from-internal",
  "priority": 1
}'
```

Ver llamadas activas desde consola Asterisk

```
sudo asterisk -rx "core show channels"
sudo asterisk -rx "pjsip show endpoints"
sudo asterisk -rx "manager show users"
```

Verificar CDR en PostgreSQL

```
SELECT calldate, src, dst, duration, disposition
FROM cdr
ORDER BY calldate DESC
LIMIT 20;
```

12. Notas para el equipo

Consideraciones de seguridad

- Los puertos AMI (5038) y ARI (8088) deben escuchar **solo en 127.0.0.1**.
- Cambiar **todos** los passwords de los archivos de ejemplo antes de cualquier despliegue.

- Las troncales SIP deben tener IP permit configurada.
- Usar TLS en SIP para troncales en producción (`transport-tls` en `pjsip.conf`).

Convenciones de código

- Toda acción sobre el AMI va en `agent/call_actions.py`, no dispersa.
- Los eventos AMI se procesan en `agent/event_handlers.py`.
- Los logs de llamadas usan `loguru` con campos estructurados: `channel`, `caller_id`, `duration`.
- No hardcodear extensiones ni contextos — van en `.env` o en tabla de configuración en BD.

Flujo de trabajo del equipo

1. Rama `develop` para features en curso.
2. Pull Request a `main` con al menos 1 revisión.
3. Migrations de BD con `alembic` (si se usa SQLAlchemy).
4. Pruebas manuales con softphone (Zoiper, MicroSIP) antes de mergear.

Recursos de referencia

- [Documentación oficial Asterisk](#)
- [AMI Actions reference](#)
- [AGI Commands](#)
- [panoramisk](#) (librería AMI Python)
- [pyst2](#) (librería AGI Python)